MIT 8103 ADVANCED DATABASE SYSTEMS · CONTINUOUS ASSESSMENT

PayNaija Wallet

Advanced Database Systems Portfolio: Full Evidence Pack

Student	Amazobi Amarachi
Student ID	301919110
Programme	Master of Information Technology, MIVA Open University
Session	2026/2027 First Semester, September Cohort
GitHub repository	https://github.com/AmazobiAmarachi/301919110-advanced-database-portfolio
Environment	Windows 11 · PostgreSQL 16 · Docker Desktop (Redis, MongoDB, Neo4j, PostgreSQL replication)
Contents	Case study · Portfolios 1–5 · AI use declaration

301919110: Advanced Database Systems Portfolio (PayNaija Wallet)

Course: MIT 8103 Advanced Database Systems, MIVA Open University (2026/2027, First Semester CA)
Student: Amazobi Amarachi · **Student ID:** 301919110 **Repository:** https://github.com/AmazobiAmarachi/301919110-advanced-database-portfolio (public)

Selected case study

PayNaija Wallet is a simulated Nigerian digital wallet and payments company (fintech) regulated by the CBN and subject to the **Nigeria Data Protection Act (NDPA) 2023**. Customers in eight states open wallet and savings accounts, fund them, pay merchants at POS terminals, buy airtime, pay bills and send money to other customers through the mobile app, USSD and the web.

Element	Description
Nature of the business	Retail digital payments: wallets, savings, P2P transfers, merchant (POS) payments
Users of the database	Customers (through the app), customer-support staff, compliance/KYC officers, administrators, fraud analysts, management (reports)
Major data stored	Customers, KYC documents, accounts and balances, cards, merchants, transfers, ledger transactions, regions, staff, audit log
Key operations	Register customer and verify KYC · open account · deposit/withdraw · **transfer funds** · **reverse transfers** · generate statements · **audit account balances** · detect fraud · regional management reports

Fintech was chosen because it stresses every topic in the course: strict **ACID** transactions for money, heavy **query workloads** on a growing ledger, **semi-structured** KYC data suited to NoSQL, **highly connected** transfer data suited to graphs, and **high-availability** requirements across data centres.

Portfolio index (all results measured on my own laptop)

#	Folder	What it demonstrates	Key result
1	`portfolio-1-database-design`	ER model, 10-table 3NF/BCNF schema, constraints, normalisation, RBAC, audit trigger	7/7 invalid operations rejected by constraints; 200,000-row ledger loaded
2	`portfolio-2-query-optimisation`	7 queries, `EXPLAIN ANALYZE` before/after, composite and partial B+ tree indexes, query rewrite, materialised view	Account statement 45.05 ms → 0.12 ms (≈366×); regional report 119 ms → 0.13 ms via materialised view
3	`portfolio-3-transactions-concurrency`	COMMIT/ROLLBACK/SAVEPOINT, lost update + fixes, isolation levels, deadlock, pgbench load test, transfer reversal, balance audit	Naive transfers: 79.4% deadlock failures (0.38 tps); ordered locking: 0% (833 tps); ledger audit difference ₦0.00
4	`portfolio-4-nosql`	Redis key-value, MongoDB document model, Neo4j fraud-detection graph (all in Docker)	COLLSCAN 20,000 docs → IXSCAN 7 docs; fraud ring 500→600→700→800→500 found on a shared device
5	`portfolio-5-distributed-cloud`	Streaming replication (Lagos → Abuja), sync vs async (CAP), failover, horizontal fragmentation, cloud design	Replay lag 2.4 ms; sync write blocked during partition (CP); primary killed → standby promoted, no committed data lost

Environment

Component	Version	Use
Windows 11 + MSYS2 UCRT64 terminal	n/a	Development environment
PostgreSQL + pgAdmin 4	16	Relational database (P1, P2, P3); pgbench load test (P3)
Docker Desktop	29.9.3	Runs the NoSQL databases (P4) and the two-site replication cluster (P5)
Redis	7 (Docker)	Key-value store (P4)
MongoDB / mongosh	7 (Docker)	Document store (P4)
Neo4j	5 (Docker)	Graph database (P4)
PostgreSQL	16 (Docker)	Primary + streaming replica (P5)
Git / GitHub	n/a	Version control, progressive commits

How to reproduce

```
createdb paynaija
psql -d paynaija -f portfolio-1-database-design/sql/01_schema.sql
psql -d paynaija -f portfolio-1-database-design/sql/02_seed.sql           # ~200,000 ledger rows (synthetic)
psql -d paynaija -f portfolio-1-database-design/sql/03_constraint_tests.sql
```

Each portfolio folder has its own README with step-by-step instructions, outputs, screenshots and analysis. All data is synthetic, generated by `02_seed.sql`; no real personal data was used.

Documentation

- `docs/Amazobi_Amarachi_301919110_Documentation.pdf`: the 3-page LMS documentation
- `docs/Amazobi_Amarachi_301919110_PortfolioEvidence.pdf`: full evidence pack (every portfolio's README, outputs and screenshots)
- `docs/AI_USE_DECLARATION.md`

Portfolio 1: Database Design and Modelling

Case study: PayNaija Wallet · **DBMS:** PostgreSQL 16 · **Course link:** Study Session 1 (stages of database design, ER model, normalisation)

1. Design process followed

The four stages of database design from Study Session 1 were applied in order:

Stage	What was done for PayNaija
1. Requirements analysis	Identified users (customers, support, compliance, admin), operations (transfer, statement, KYC, audit) and business rules (below)
2. Conceptual design	ER diagram with 10 entities and their cardinalities (Figure 1)
3. Logical design	ER mapped to relations; PK/FK chosen; normalised to 3NF/BCNF (Section 4)
4. Physical design	Data types (`NUMERIC(15,2)` for money), CHECK/UNIQUE constraints, trigger, roles; indexes added in Portfolio 2

Business rules captured as constraints

- A customer must be 18 or older and have a unique email and a unique Nigerian phone number (`+234` + 10 digits).
- A customer may own many accounts; every account belongs to exactly one customer.
- A balance can never be negative (no overdraft on wallets).
- A transfer has exactly one sender and one receiver account, and they must be different.
- A transfer posts exactly two ledger rows: a debit (DR) and a credit (CR), giving double-entry book-keeping.
- KYC documents are verified by a staff member; a VERIFIED document must record who verified it and when.
- A customer who still has accounts cannot be deleted (protects financial history).
- Every balance change must be traceable (audit log, for NDPA 2023 and CBN audit requirements).

2. Entity-Relationship diagram

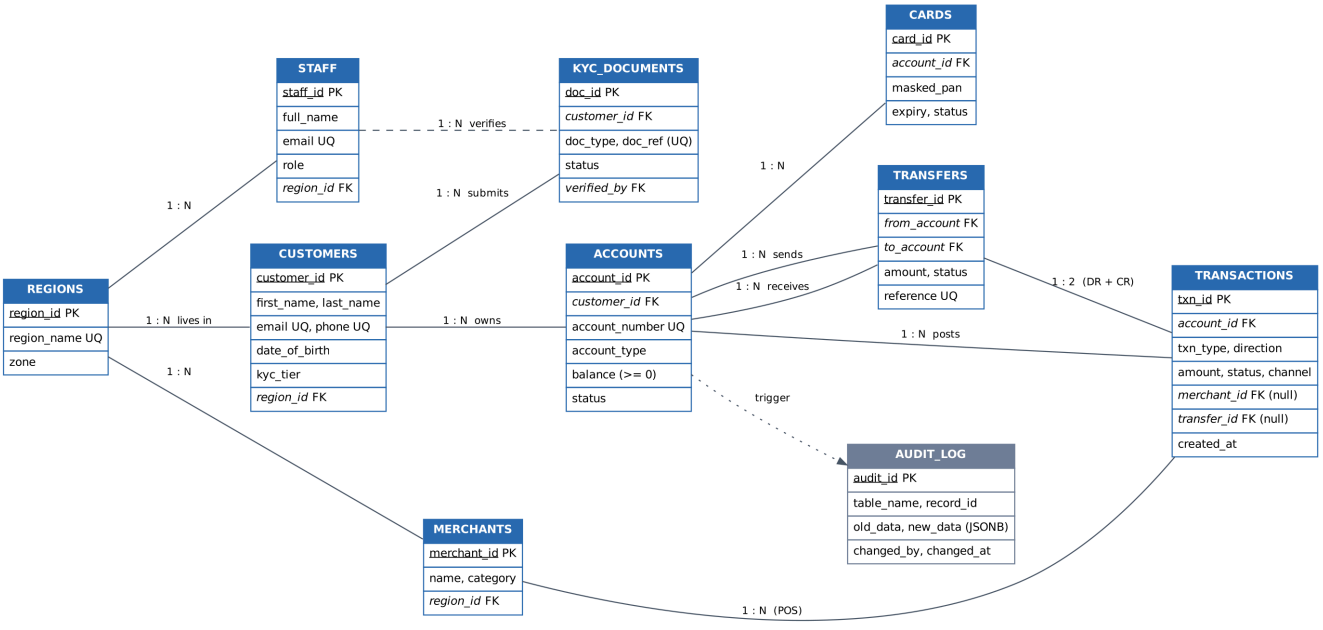


Figure 1. ER diagram (source `erd.dot`, rendered with Graphviz). PK = primary key, FK = foreign key, UQ = unique.

Key relationships

- `CUSTOMERS 1:N ACCOUNTS`, `CUSTOMERS 1:N KYC_DOCUMENTS`, `REGIONS 1:N CUSTOMERS`
- `ACCOUNTS 1:N TRANSACTIONS` (the ledger)
- **`ACCOUNTS M:N ACCOUNTS`** (an account can send to and receive from many accounts). This recursive many-to-many relationship is resolved by the associative entity **`TRANSFERS`**, which has two foreign keys to `ACCOUNTS` (`from_account`, `to_account`).
- `STAFF 1:N KYC_DOCUMENTS` (verifies), `MERCHANTS 1:N TRANSACTIONS` (optional; POS only)

3. Relational schema

```
REGIONS(region_id PK, region_name UQ, zone)
STAFF(staff_id PK, full_name, email UQ, role, region_id FK→REGIONS)
CUSTOMERS(customer_id PK, first_name, last_name, email UQ, phone UQ, date_of_birth, region_id FK→REGIONS,
kyc_tier, created_at)
KYC_DOCUMENTS(doc_id PK, customer_id FK→CUSTOMERS, doc_type, doc_ref, status, verified_by FK→STAFF,
verified_at)   UQ(doc_type, doc_ref)
ACCOUNTS(account_id PK, customer_id FK→CUSTOMERS, account_number UQ, account_type, balance, status, opened_at)
CARDS(card_id PK, account_id FK→ACCOUNTS, masked_pan, expiry, status)
MERCHANTS(merchant_id PK, name, category, region_id FK→REGIONS)
TRANSFERS(transfer_id PK, from_account FK→ACCOUNTS, to_account FK→ACCOUNTS, amount, status, reference UQ,
created_at)
TRANSACTIONS(txn_id PK, account_id FK→ACCOUNTS, txn_type, direction, amount, status, merchant_id FK→MERCHANTS,
transfer_id FK→TRANSFERS, channel, created_at)
AUDIT_LOG(audit_id PK, table_name, record_id, action, old_data JSONB, new_data JSONB, changed_by, changed_at)
```

Full DDL: `sql/01_schema.sql`. Selected design decisions:

Decision	Reason
`NUMERIC(15,2)` for money, never `FLOAT`	Floating point cannot represent ₦0.10 exactly, and rounding errors are unacceptable in a ledger
Surrogate keys (`SERIAL`) plus natural keys kept `UNIQUE` (email, phone, account_number)	Stable, small join keys; business identifiers can change (e.g. a phone number) without cascading updates
`CHECK (balance >= 0)`	Enforces "no overdraft" inside the database, so no application bug can bypass it
`ON DELETE RESTRICT` on accounts→customers	Financial records must not disappear when a customer is deleted
`CHECK (from_account <> to_account)`	Prevents meaningless self-transfers
`masked_pan` stores only first 6 + last 4 digits	PCI-DSS and NDPA data minimisation: the database never stores full card numbers
`AFTER UPDATE` trigger writing to `audit_log` (JSONB old/new)	Accountability principle (NDPA 2023); shows who changed which balance and when
Roles `support_role` (read customer data only) and `compliance_role` (read everything); `audit_log` revoked from support	Least privilege / RBAC (Study Session 6)

4. Normalisation

The starting point is the "customer statement" spreadsheet that the operations team used before the database existed (**UNF**):

customer_id	name	phones	region	zone	accounts (repeating) → transactions (repeating)
3	Halima Adeyemi	07000000003, 08031112222	Lagos	South-West	3000000003 WALLET ₦57,569 → {T1, 2026-09-12, ₦5,000, Merchant 7, Fuel}, {T2, ...} ; 4000000003 SAVINGS ...

UNF → 1NF (atomic values, no repeating groups). Phones are split into single values, and the repeating account and transaction groups are flattened to one row per transaction:

`R1(customer_id, name, phone, region, zone, account_number, account_type, balance, txn_no, txn_date, amount, merchant_name, merchant_category)`, key = **(account_number, txn_no)**

Functional dependencies identified:

- FD1 `account_number → account_type, balance, customer_id`
- FD2 `customer_id → name, phone, region`
- FD3 `region → zone`
- FD4 `(account_number, txn_no) → txn_date, amount, merchant_name`
- FD5 `merchant_name → merchant_category`

Anomalies in 1NF: the balance is repeated on every transaction row (update anomaly); a new account cannot be stored until it has a transaction (insertion anomaly); deleting a customer's only transaction deletes the account (deletion anomaly).

1NF → 2NF (remove partial dependencies). FD1 depends on only *part* of the key (`account_number`), so those attributes move out:

- `ACCOUNT(account_number, account_type, balance, customer_id, name, phone, region, zone)`
- `TXN(account_number, txn_no, txn_date, amount, merchant_name, merchant_category)`

2NF → 3NF (remove transitive dependencies). In ACCOUNT, `account_number → customer_id → name, phone, region → zone` is transitive. In TXN, `… → merchant_name → merchant_category` is transitive. Splitting gives:

- `REGION(region_id, region_name, zone)`
- `CUSTOMER(customer_id, name, phone, region_id)`
- `ACCOUNT(account_number, account_type, balance, customer_id)`
- `MERCHANT(merchant_id, name, category)`
- `TRANSACTION(txn_id, account_id, date, amount, merchant_id)`

BCNF check. In every final table, every determinant is a candidate key (e.g. in CUSTOMERS both `email` and `phone` determine the row and are declared UNIQUE, so they are candidate keys). The schema is therefore in **BCNF**.

Deliberate exception (justified denormalisation). `accounts.balance` can be derived from the sum of the ledger, so storing it is technically redundant. It is kept because recomputing a balance from thousands of rows on every transfer would be too slow (physical design trade-off). Consistency is protected by updating the balance and the ledger in the **same ACID transaction** (Portfolio 3) and by the audit trigger.

5. Implementation and test evidence

Data loaded (`sql/02_seed.sql`; synthetic, reproducible with `setseed`, no real personal data):

```
       t       | count
---------------+--------
 regions       |      8
 customers     |   2000
 kyc_documents |   2000
 accounts      |   2987
 merchants     |     50
 cards         |   2000
 transactions  | 200000
```

Constraint tests (`sql/03_constraint_tests.sql`). Every invalid operation is rejected by the DBMS:

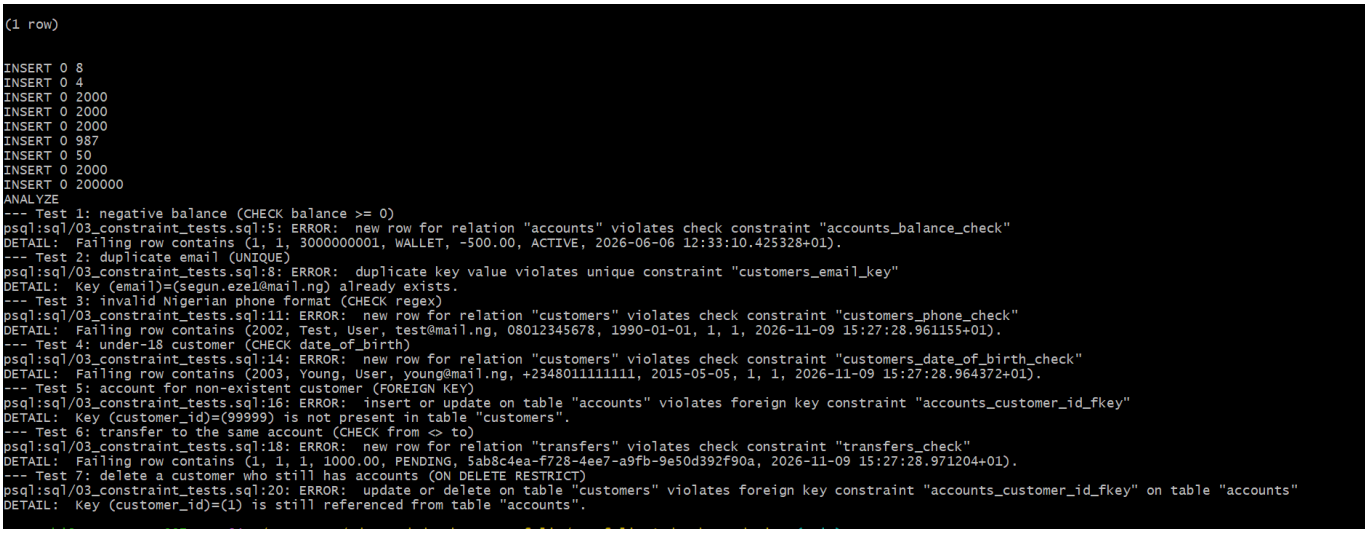
```
--- Test 1: negative balance (CHECK balance >= 0)
ERROR:  new row for relation "accounts" violates check constraint "accounts_balance_check"
--- Test 2: duplicate email (UNIQUE)
ERROR:  duplicate key value violates unique constraint "customers_email_key"
--- Test 3: invalid Nigerian phone format (CHECK regex)
ERROR:  new row for relation "customers" violates check constraint "customers_phone_check"
--- Test 4: under-18 customer (CHECK date_of_birth)
ERROR:  new row for relation "customers" violates check constraint "customers_date_of_birth_check"
--- Test 5: account for non-existent customer (FOREIGN KEY)
ERROR:  insert or update on table "accounts" violates foreign key constraint "accounts_customer_id_fkey"
--- Test 6: transfer to the same account (CHECK from <> to)
ERROR:  new row for relation "transfers" violates check constraint "transfers_check"
--- Test 7: delete a customer who still has accounts (ON DELETE RESTRICT)
ERROR:  update or delete on table "customers" violates foreign key constraint "accounts_customer_id_fkey"
```

Full output: `results/03_constraint_tests_output.txt`

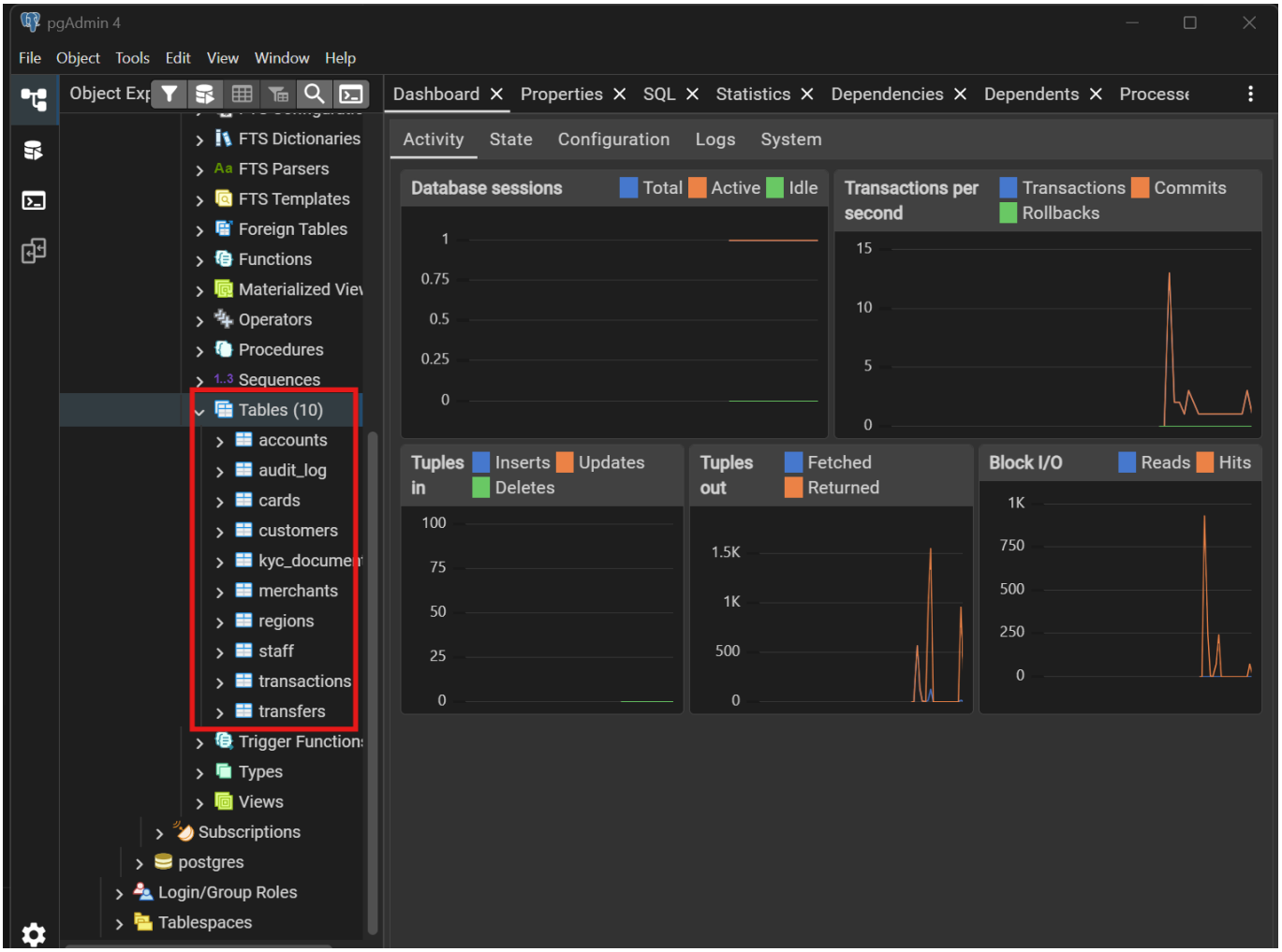
6. How to run

```
createdb paynaija
psql -d paynaija -f sql/01_schema.sql
psql -d paynaija -f sql/02_seed.sql
psql -d paynaija -f sql/03_constraint_tests.sql
```

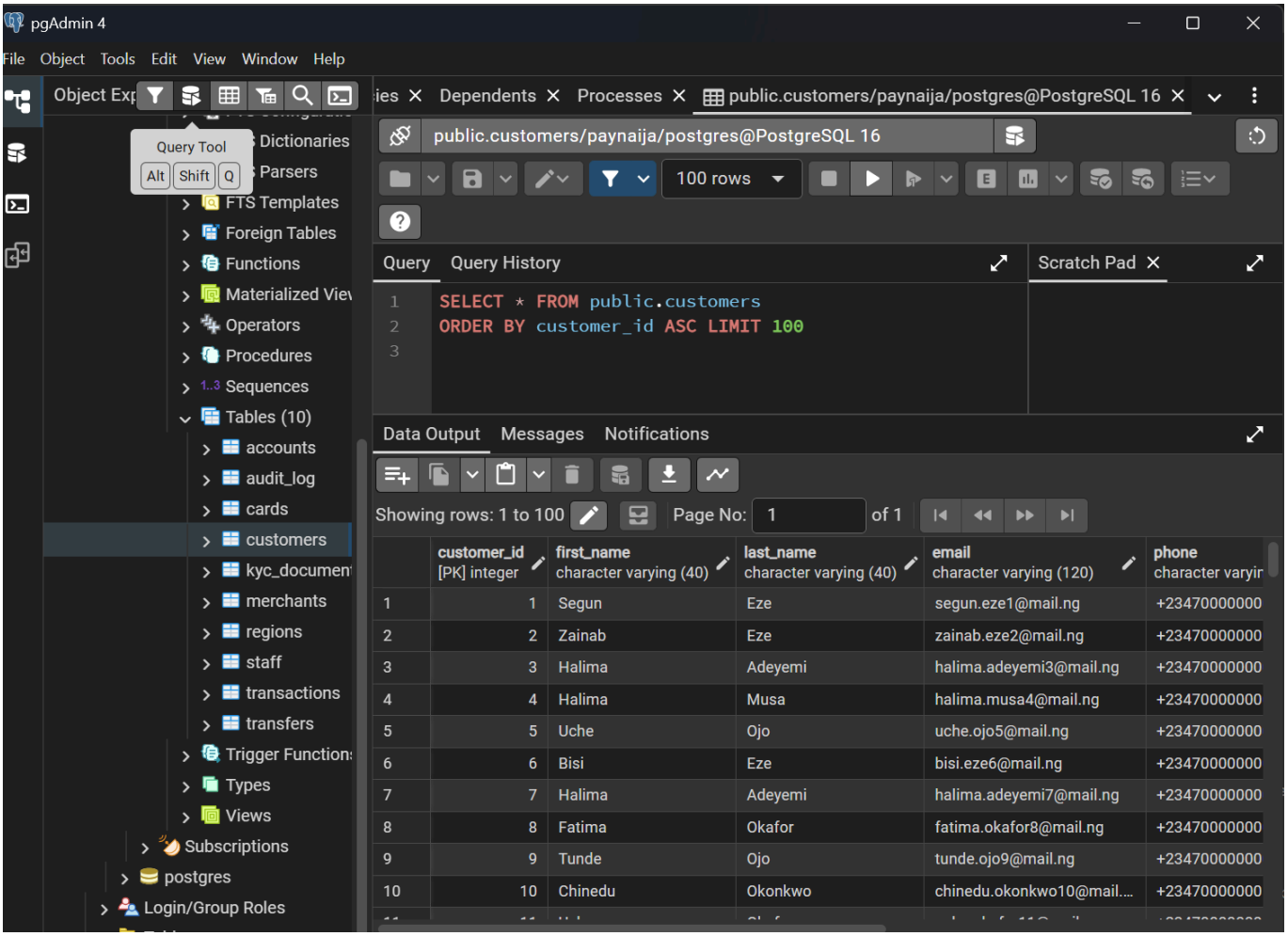
7. Screenshots from my environment



All 7 invalid operations rejected by PostgreSQL on my laptop.



The 10 PayNaija tables in pgAdmin 4.



Sample customer records loaded by 02_seed.sql.

Portfolio 2: Query Processing and Optimisation

Case study: PayNaija Wallet (200,000-row ledger) · **DBMS:** PostgreSQL 16 on Windows 11 (cost-based optimiser) · **Course link:** Study Session 4

1. Method

1. Wrote 7 business queries (`sql/queries/`) that represent real PayNaija workloads, from simple OLTP lookups to OLAP-style reports.
2. **Baseline:** removed all optional indexes (`sql/01_drop_indexes.sql`) and ran `ANALYZE` to refresh statistics. Each query was then run **5 times** with `EXPLAIN (ANALYZE, BUFFERS)`, keeping the **median** execution time (`run_benchmark.sh`). The median avoids distortion from the first "cold cache" run.
3. Read the BEFORE plans to find the bottleneck operator in each query.
4. **Optimised** with indexes chosen from those plans (`sql/02_create_indexes.sql`), a query rewrite (Q3) and a materialised view (Q4, `sql/03_materialised_view.sql`).
5. Re-ran the identical benchmark (AFTER) and compared the two. All plans are saved in `results/plans_before/` and `results/plans_after/`, and the raw timings in `results/timings_before.csv` and `results/timings_after.csv`.

2. Queries

ID	Business purpose	SQL features
Q1	Account statement, last 90 days (most frequent app call)	filter + range + ORDER BY
Q2	Customer statement by phone number	3-table JOIN
Q3	Failed transactions on a given day (operations report)	function on column vs range predicate
Q4	Monthly debit value per region (management report)	4-table JOIN, GROUP BY, aggregation
Q5	Top 3 spenders in each region	CTE + window function `rank() OVER (PARTITION BY …)`
Q6	Fraud check: accounts with ≥3 failed attempts in 30 days	GROUP BY + HAVING

3. Optimisations applied

```
-- Composite B+ tree: equality on account_id, then range + sort on created_at  (Q1, Q2)
CREATE INDEX idx_txn_account_created ON transactions (account_id, created_at DESC);
-- Partial covering index: only FAILED rows (~3%), so it is small and fast       (Q3, Q6)
CREATE INDEX idx_txn_failed_created ON transactions (created_at)
       INCLUDE (account_id, amount, channel) WHERE status = 'FAILED';
-- Foreign-key index (PostgreSQL does not create these automatically)          (Q2, Q5)
CREATE INDEX idx_accounts_customer ON accounts (customer_id);
```

Index	Why this design
`idx_txn_account_created`	Column order matters: the **equality** column comes first (`account_id`), then the **range/sort** column.
`idx_txn_failed_created`	A **partial** index stores only the ~3% of rows that are FAILED, so it is much smaller than a full index. `INCLUDE` makes it **covering**, so Q6 can be answered from the index alone (Index Only Scan).
`idx_accounts_customer`	Lets the optimiser use an index nested-loop join from customers to accounts.

Query rewrite (Q3). `WHERE created_at::date = CURRENT_DATE - 7` wraps the column in a function. The B+ tree is ordered by `created_at`, not by `created_at::date`, so the index cannot be used for the date condition: the predicate is not *sargable*. It was rewritten as the equivalent range `created_at >= CURRENT_DATE - 7 AND created_at < CURRENT_DATE - 6`.

Materialised view (Q4). The monthly/regional summary (104 rows) is pre-computed and refreshed nightly with `REFRESH MATERIALIZED VIEW CONCURRENTLY`.

4. Results (median of 5 runs, measured on my laptop)

Query	Before (ms)	After (ms)	Plan change	Speed-up
Q1 account statement	45.05	0.12	Parallel **Seq Scan** + Sort → **Bitmap Index Scan** on `idx_txn_account_created`	**≈ 366×**
Q2 customer statement (join)	46.86	0.47	Full scan of `transactions` → **Nested Loop** of three **Index Scans**	**≈ 99×**
Q3 original (function on column)	47.43	5.65	Partial index used for `status` only; date still filtered row by row	8×
Q3 rewritten (range)	48.32	0.16	Both conditions resolved inside the partial index	**≈ 312×**
Q4 regional report (live)	119.17	119.97	**No change**: still a full scan + hash joins + aggregation	1×
Q4b regional report (materialised view)	119.17	0.13	Scan of a 104-row pre-computed view	**≈ 931×**
Q5 top spenders (window)	72.16	69.31	**No real change**: reads most of the table either way	≈ 1×
Q6 fraud velocity	45.72	0.37	Parallel Seq Scan → index scan on the partial index	**≈ 123×**

Evidence: Q1 execution plan before and after

```
$ cat results/plans_before/Q1_account_statement.txt
Gather Merge  (cost=5616.09..5617.13 rows=9 width=45) (actual time=42.474..46.548 rows=12 loops=1)
  Workers Planned: 1
  Workers Launched: 1
  Buffers: shared hit=2300
  ->  Sort  (cost=4616.08..4616.11 rows=9 width=45) (actual time=8.191..8.193 rows=6 loops=2)
        Sort Key: created_at DESC
        Sort Method: quicksort  Memory: 25kB
        Buffers: shared hit=2300
        Worker 0:  Sort Method: quicksort  Memory: 25kB
        ->  Parallel Seq Scan on transactions  (cost=0.00..4615.94 rows=9 width=45) (actual time=0.332..8.162 rows=6 loops=2)
              Filter: ((account_id = 1450) AND (created_at >= (now() - '90 days'::interval)))
              Rows Removed by Filter: 99994
              Buffers: shared hit=2263
Planning:
  Buffers: shared hit=109
Planning Time: 0.336 ms
Execution Time: 46.585 ms
```

```
BEFORE  (Execution Time: 46.585 ms)
Gather Merge  (cost=5616.09..5617.13 rows=9) (actual time=42.474..46.548 rows=12 loops=1)
  Workers Planned: 1   Workers Launched: 1
  ->  Sort  (actual time=8.191..8.193 rows=6 loops=2)   Sort Key: created_at DESC
        ->  Parallel Seq Scan on transactions  (cost=0.00..4615.94 rows=9) (actual time=0.332..8.162 rows=6
loops=2)
              Filter: ((account_id = 1450) AND (created_at >= (now() - '90 days'::interval)))
              Rows Removed by Filter: 99994
              Buffers: shared hit=2263
```

```
$ cat results/plans_after/Q1_account_statement.txt
Sort  (cost=65.19..65.23 rows=16 width=45) (actual time=0.091..0.091 rows=12 loops=1)
  Sort Key: created_at DESC
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=18
  ->  Bitmap Heap Scan on transactions  (cost=4.59..64.87 rows=16 width=45) (actual time=0.025..0.073 rows=12 loops=1)
        Recheck Cond: ((account_id = 1450) AND (created_at >= (now() - '90 days'::interval)))
        Heap Blocks: exact=12
        Buffers: shared hit=15
        ->  Bitmap Index Scan on idx_txn_account_created  (cost=0.00..4.58 rows=16 width=0) (actual time=0.018..0.018 rows=12 loops=1)
              Index Cond: ((account_id = 1450) AND (created_at >= (now() - '90 days'::interval)))
              Buffers: shared hit=3
Planning:
  Buffers: shared hit=165
Planning Time: 0.455 ms
Execution Time: 0.126 ms
```

```
AFTER   (Execution Time: 0.126 ms)
Sort  (cost=65.19..65.23 rows=16) (actual time=0.091..0.091 rows=12 loops=1)
  Buffers: shared hit=18
  ->  Bitmap Heap Scan on transactions  (cost=4.59..64.87 rows=16) (actual time=0.025..0.073 rows=12)
        Heap Blocks: exact=12
        ->  Bitmap Index Scan on idx_txn_account_created  (cost=0.00..4.58 rows=16) (actual time=0.018..0.018
rows=12)
              Index Cond: ((account_id = 1450) AND (created_at >= (now() - '90 days'::interval)))
```

The optimiser's estimated cost fell from **5,617 to 65 units**, and the pages read fell from **2,300 to 18** buffers. The planner estimated 16 matching rows and the actual number was 12, which shows the table statistics were accurate.

Evidence: Q2, the optimiser changed the join algorithm

```
$ cat results/plans_after/Q2_customer_statement_join.txt
Sort  (cost=24.96..25.13 rows=67 width=46) (actual time=0.421..0.425 rows=122 loops=1)
  Sort Key: t.created_at DESC
  Sort Method: quicksort  Memory: 34kB
  Buffers: shared hit=139
  ->  Nested Loop  (cost=0.98..22.93 rows=67 width=46) (actual time=0.053..0.382 rows=122 loops=1)
        Buffers: shared hit=136
        ->  Nested Loop  (cost=0.56..16.60 rows=1 width=27) (actual time=0.035..0.037 rows=2 loops=1)
              Buffers: shared hit=7
              ->  Index Scan using customers_phone_key on customers c  (cost=0.28..8.29 rows=1 width=16) (actual time=0.021..0.021 rows=1 loops=1)
                    Index Cond: ((phone)::text = '+2347000001234'::text)
                    Buffers: shared hit=3
              ->  Index Scan using idx_accounts_customer on accounts a  (cost=0.28..8.30 rows=1 width=19) (actual time=0.011..0.013 rows=2 loops=1)
                    Index Cond: (customer_id = c.customer_id)
                    Buffers: shared hit=4
        ->  Index Scan using idx_txn_account_created on transactions t  (cost=0.42..5.66 rows=67 width=27) (actual time=0.012..0.165 rows=61 loops=2)
              Index Cond: (account_id = a.account_id)
              Buffers: shared hit=129
Planning:
  Buffers: shared hit=400
Planning Time: 1.122 ms
Execution Time: 0.458 ms
```

```
AFTER  (Execution Time: 0.458 ms)
Sort  (cost=24.96..25.13 rows=67) (actual rows=122)
  ->  Nested Loop  (actual rows=122)
        ->  Nested Loop  (actual rows=2)
              ->  Index Scan using customers_phone_key on customers c       (actual rows=1)
              ->  Index Scan using idx_accounts_customer on accounts a      (actual rows=2)
        ->  Index Scan using idx_txn_account_created on transactions t      (actual rows=61 loops=2)
```

The query finds **1 customer** by phone (unique index), then that customer's **2 accounts** (FK index), then **61 transactions per account** (composite index). The whole statement touched only 139 buffers.

The cost of indexes (write overhead)

`sql/04_index_write_cost.sql` inserts 50,000 ledger rows inside a rolled-back transaction (so no data is kept), 3 runs each:

Scenario	Run 1	Run 2	Run 3	Median
With the 2 extra transaction indexes	471.7 ms	471.1 ms	567.9 ms	**471.7 ms**
Without them	357.6 ms	364.9 ms	391.8 ms	**364.9 ms**

Inserts were **about 29% slower** with the indexes, because every new row must also be added to each B+ tree.

5. Analysis

Why the OLTP queries improved. PostgreSQL uses **cost-based optimisation (CBO)**: it estimates the cost of alternative plans from table statistics collected by `ANALYZE` and picks the cheapest. Before indexing, the only way to find one account's transactions was a full sequential scan, so Q1 examined 200,000 rows (99,994 removed by the filter in each of two processes) to return 12. With the composite index, the optimiser predicted a highly **selective** result (16 rows) and switched to an index scan costing 65 units instead of 5,617. In Q2 the cheaper access paths also changed the **join strategy**: with an index on every join column, a **nested-loop join** became cheapest, because each step starts from a tiny input (1 customer → 2 accounts) and uses an index to find the matching rows.

An observation from my Windows environment. In the Q1 BEFORE plan, the scan and sort inside the parallel worker finished in about **8 ms**, but the top `Gather Merge` node only finished at **46.5 ms**. Most of the baseline time (around 38 ms) was the overhead of **launching a parallel worker**. PostgreSQL on Windows has to start a new process for each worker, which is more expensive than on Linux. This is why my baseline times (about 45 ms) are higher than results reported on Linux for the same data, while my indexed times (about 0.1 ms) are almost identical. **Index lookups are far less sensitive to the platform than full scans**, so the benefit of indexing was even larger on my machine (366× for Q1).

Heuristics from the course seen in practice. The planner applied **selection push-down** (filters evaluated inside the scan node, never after a join), **projection** (only the needed columns move up the plan), and joins instead of Cartesian products, all automatically.

Why Q4 and Q5 did not improve, and why that is correct. These are **OLAP** queries that need most of the 200,000 rows (very low selectivity). Fetching that many rows through an index would cost more random I/O than one sequential pass, so the CBO correctly ignored the new indexes. This matches the course point that row-store RDBMS are optimised for OLTP rather than analytics. The right tool was **pre-aggregation**: the materialised view answered the Q4 report in 0.13 ms (≈ 931× faster). Its trade-off is **staleness**: the data is only as fresh as the last refresh. That is acceptable for a management report but never for balances. A larger PayNaija would move such reports to a data warehouse (Session 3).

Sargability. Q3 showed that even when the right index exists, wrapping the indexed column in a function (`created_at::date`) stops the index being used for that condition (5.65 ms). Rewriting it as a range predicate made the same query **36× faster again** (0.16 ms), with no new index needed.

Trade-offs. Indexes are not free: inserts were about 29% slower, and every index uses extra disk space and must be maintained. PayNaija's workload is read-heavy for statements and fraud checks, so the trade is worthwhile. Only three targeted indexes were created rather than indexing every column, and the fraud index is partial, so it stays small.

6. How to run

```
psql -d paynaija -f sql/01_drop_indexes.sql && bash run_benchmark.sh before
psql -d paynaija -f sql/02_create_indexes.sql && psql -d paynaija -f sql/03_materialised_view.sql && bash
run_benchmark.sh after
psql -d paynaija -f sql/04_index_write_cost.sql
```

(Q4b reports an error in the BEFORE run because the materialised view does not exist yet. This is expected.)

Portfolio 3: Transactions and Concurrency

Case study: PayNaija Wallet fund transfers · **DBMS:** PostgreSQL 16 on Windows 11 (MVCC, default isolation READ COMMITTED) · **Course link:** ACID / BASE (Study Session 2), concurrency and locking issues (Session 1)

All demos use accounts **101** and **102**, reset to ₦50,000 each before every demo. Concurrency demos run **two real database sessions at the same time** (Session A and Session B) using `run_concurrent.sh`; timestamps come from `clock_timestamp()`.

Part A: Transaction control (single session)

Demo 1: COMMIT: successful ₦10,000 transfer (`sql/01_commit_transfer.sql`)

Four statements (transfer record, debit, credit, two ledger rows) run as **one atomic unit**.

```
COMMIT
 account_id | balance
        101 | 40000.00
        102 | 60000.00
audit_log (latest 2 rows):
 record_id 102 | old {"balance": 50000.00} | new {"balance": 60000.00}
 record_id 101 | old {"balance": 50000.00} | new {"balance": 40000.00}
```

Full output: `results/01_commit_transfer_output.txt` The audit trigger from Portfolio 1 recorded both balance changes inside the same transaction.

Demo 2: ROLLBACK: insufficient funds (`sql/02_rollback_insufficient_funds.sql`)

Transfer ₦100,000 when account 101 holds only ₦40,000. The **credit to 102 runs first and succeeds**, then the debit violates `CHECK (balance >= 0)`:

```
UPDATE accounts SET balance = balance + 100000 WHERE account_id = 102;   -> UPDATE 1
UPDATE accounts SET balance = balance - 100000 WHERE account_id = 101;
ERROR:  new row for relation "accounts" violates check constraint "accounts_balance_check"
COMMIT;   -> ROLLBACK
 account_id | balance
        101 | 40000.00      <- unchanged
        102 | 60000.00      <- the ₦100,000 credit was undone
```

Atomicity: PostgreSQL refused to commit a half-finished transfer, so no money was created. **Consistency:** the constraint guaranteed the database moved only between valid states.

Demo 3: SAVEPOINT (partial rollback) (`sql/03_savepoint.sql`)

A ₦2,000 transfer succeeds, but posting an optional ₦50 fee fails (FK error). `ROLLBACK TO SAVEPOINT before_fee` undoes only the fee, and the transfer commits: 101 = ₦38,000, 102 = ₦62,000.

Demo 4: Stored procedure `transfer_funds()` (`sql/04_transfer_procedure.sql`)

Production version used by the rest of the portfolio. It (1) locks both account rows with `SELECT … FOR UPDATE` **in ascending account_id order**, (2) validates status and funds, (3) writes the transfer and both ledger entries.

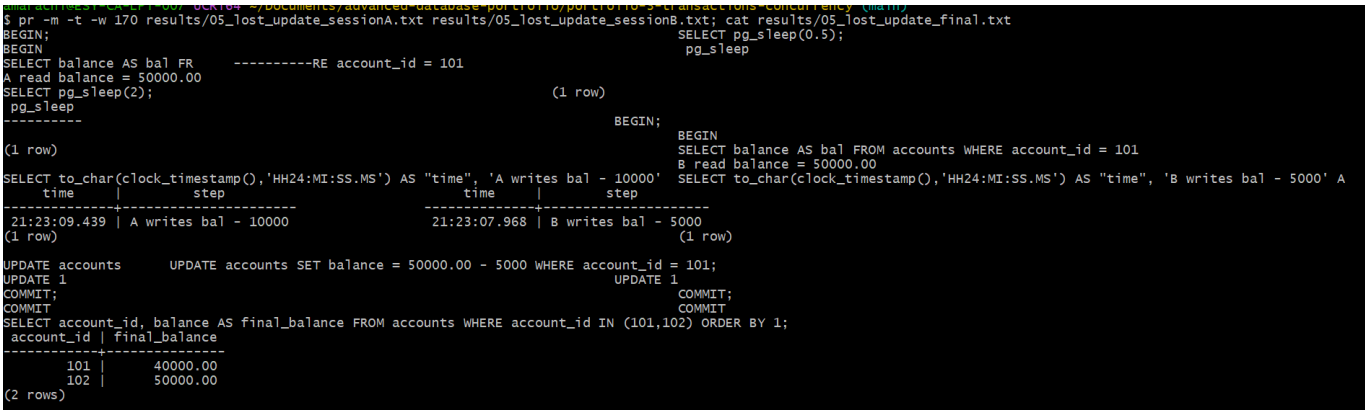
```
CALL transfer_funds(101, 102, 5000);      -> CALL
CALL transfer_funds(101, 102, 9000000);
ERROR:  Insufficient funds: balance 33000.00, requested 9000000
```

Part B: Concurrency problems and control mechanisms (two sessions)

Demo 5: The problem: LOST UPDATE (`sql/concurrency/05_lost_update_A.sql` / `_B.sql`)

Two app servers withdraw from account 101 (₦50,000) at the same time using read-then-write logic with no lock.

Time	Session A (withdraw ₦10,000)	Session B (withdraw ₦5,000)
t = 0 s	`BEGIN` ; reads balance = **50,000**	
t = 0.5 s	(busy: fraud check / network call)	`BEGIN` ; reads balance = **50,000**
t ≈ 0.5 s		writes 50,000 − 5,000 = **45,000**; `COMMIT`
t ≈ 2 s	writes 50,000 − 10,000 = **40,000**; `COMMIT` (overwrites B)	



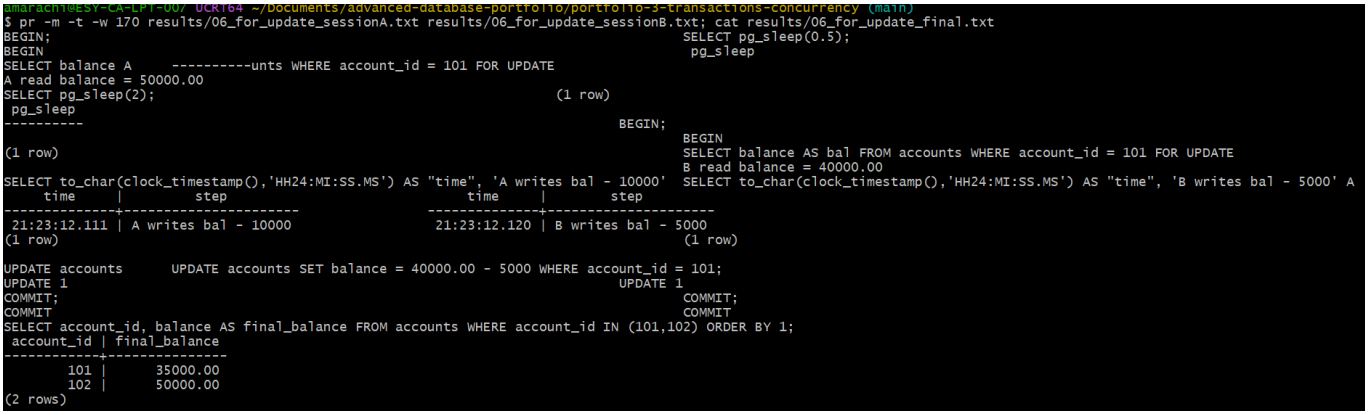

```
  account_id | final_balance
         101 |      40000.00        <- should be 35,000. B's ₦5,000 withdrawal was LOST
```

The customer received ₦15,000 but was charged only ₦10,000. Both transactions committed with no error, which is why this anomaly is dangerous.

Demo 6: Fix 1: pessimistic locking with `SELECT … FOR UPDATE`

Same scripts, but the read takes a row-level **exclusive lock**:

```
 Session A:  SELECT balance ... FOR UPDATE   -> 50000.00   ... writes 40000, COMMIT
 Session B:  SELECT balance ... FOR UPDATE   -> BLOCKED until A commits, then reads 40000.00
             writes 40000 - 5000, COMMIT
  account_id | final_balance
         101 |      35000.00        <- correct
```



B waited about 1.5 s and then saw A's committed value. The two transactions were serialised, as in **two-phase locking (2PL)**: locks are held until commit.

Demo 7: Fix 2: REPEATABLE READ isolation (optimistic)

A updates 101 and holds its transaction open; B (REPEATABLE READ) reads 50,000 from its snapshot and then tries to update the same row:

```
Session B: UPDATE accounts SET balance = balance - 5000 WHERE account_id = 101;
ERROR:  could not serialize access due to concurrent update        (SQLSTATE 40001)
B: application catches SQLSTATE 40001 and RETRIES the whole transaction:
UPDATE 1 / COMMIT
 final_balance = 35000.00                                            <- correct
```

Instead of blocking, the database **detects** the conflict and forces B to retry with fresh data. This is how MVCC databases avoid lost updates without read locks.

Demo 8: Isolation levels: non-repeatable read

Session A reads account 101 twice, 2 s apart; Session B commits a ₦1,000 withdrawal in between.

Isolation level of A	First read	Second read	Anomaly?
READ COMMITTED (PostgreSQL default)	50,000.00	**49,000.00**	**Yes**: non-repeatable read
REPEATABLE READ	50,000.00	**50,000.00**	No: A keeps its snapshot

Demo 9: Deadlock (`sql/concurrency/09_deadlock_A.sql` / `_B.sql`)

A transfers 101→102 (locks 101, then wants 102); B transfers 102→101 (locks 102, then wants 101).

```
Session A: UPDATE accounts ... WHERE account_id = 102;
ERROR:  deadlock detected
DETAIL:  Process 35384 waits for ShareLock on transaction 916; blocked by process 33552.
         Process 33552 waits for ShareLock on transaction 915; blocked by process 35384.
COMMIT; -> ROLLBACK          (A is chosen as the victim)
Session B: UPDATE 1 / COMMIT (B completes)
 account_id 101 final_balance = 51000.00   <- only B's 102 -> 101 transfer of N1,000 survived
```

```
$ cat results/09_deadlock_sessionA.txt results/09_deadlock_final.txt
BEGIN;
BEGIN
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 101;
UPDATE 1
SELECT pg_sleep(1);
 pg_sleep
----------

(1 row)

SELECT to_char(clock_timestamp(),'HH24:MI:SS.MS') AS "time", 'A now wants row 102' AS step;
     time     |        step
--------------+--------------------
 21:23:22.210 | A now wants row 102
(1 row)

UPDATE accounts SET balance = balance + 1000 WHERE account_id = 102;
psql:sql/concurrency/09_deadlock_A.sql:6: ERROR:  deadlock detected
DETAIL:  Process 35384 waits for ShareLock on transaction 916; blocked by process 33552.
Process 33552 waits for ShareLock on transaction 915; blocked by process 35384.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (30,85) in relation "accounts"
COMMIT;
ROLLBACK
SELECT account_id, balance AS final_balance FROM accounts WHERE account_id IN (101,102) ORDER BY 1;
 account_id | final_balance
------------+---------------
        101 |      51000.00
        102 |      49000.00
(2 rows)
```

After `deadlock_timeout` (1 s), PostgreSQL's deadlock detector found the cycle in the **wait-for graph** and aborted one transaction so the other could finish.

Demo 10: Load test: deadlock prevention at scale (`run_stress_test.sh`)

`pgbench`, **8 concurrent clients × 40 transfers**, random direction between 101 and 102:

Implementation	Processed	Deadlock failures	Throughput	Money conserved (101 + 102)
Naive: lock rows in transfer order	66 / 320	**254 (79.4%)**	0.38 tps	₦100,000 ✔
`transfer_funds()` : lock rows in **ascending id order**	320 / 320	**0 (0%)**	**833.3 tps**	₦100,000 ✔

```
$ cat results/10_stress_test_output.txt
===== naive_transfer
number of clients: 8
number of transactions actually processed: 66/320
number of failed transactions: 254 (79.375%)
number of serialization failures: 0 (0.000%)
number of deadlock failures: 254 (79.375%)
tps = 0.375851 (without initial connection time)
 total_money_101_plus_102
--------------------------
                100000.00
(1 row)

===== procedure_transfer
number of clients: 8
number of transactions actually processed: 320/320
number of failed transactions: 0 (0.000%)
number of serialization failures: 0 (0.000%)
number of deadlock failures: 0 (0.000%)
tps = 833.274744 (without initial connection time)
 total_money_101_plus_102
--------------------------
                100000.00
(1 row)
```

Analysis

Isolation level	Dirty read	Non-repeatable read	Phantom	Lost update	In PostgreSQL
Read Uncommitted	possible*	possible	possible	possible	treated as Read Committed
Read Committed (default)	prevented	**possible (Demo 8)**	possible	**possible (Demo 5)**	
Repeatable Read	prevented	**prevented (Demo 8)**	prevented in PG	**detected (Demo 7)**	snapshot isolation
Serializable	prevented	prevented	prevented	prevented	SSI, may abort with 40001

*PostgreSQL never allows dirty reads, because of MVCC.

- **ACID in practice.** Demo 2 showed *atomicity* (all-or-nothing), the CHECK constraint enforced *consistency*, Demos 5–8 tested *isolation*, and the committed transfer survived the crash and failover in Portfolio 5 (*durability*, via the write-ahead log).

- **Pessimistic vs optimistic control.** `FOR UPDATE` (Demo 6) makes the second transaction **wait**; REPEATABLE READ (Demo 7) lets it proceed and **aborts** it on conflict, so the application must retry. For a hot wallet account that is updated very often, pessimistic locking avoids repeated retries. For rarely-conflicting updates, optimistic control gives higher concurrency.
- **Deadlocks are a design problem, not a database bug.** The detector *resolves* deadlocks, but each one costs a 1-second wait plus a failed customer transfer: throughput fell to 0.38 tps and 79% of transfers failed. Enforcing a **global lock order** (always lock the lower account_id first) *prevents* the circular wait, which is one of the four Coffman conditions for deadlock. The result was zero deadlocks and about 2,200× higher throughput (833 vs 0.38 tps), with the same money-conservation guarantee.
- **Choice for PayNaija:** keep READ COMMITTED (fast) as the default, and make every money movement go through `transfer_funds()`, which takes explicit row locks in a fixed order. Use SERIALIZABLE with retry for complex compliance batch jobs.

How to run

```
psql -d paynaija -f sql/00_setup_demo_accounts.sql
for f in 01_commit_transfer 02_rollback_insufficient_funds 03_savepoint 04_transfer_procedure; do psql -d
paynaija -e -f sql/$f.sql; done
for d in 05_lost_update 06_for_update 07_repeatable_read 08_nonrepeatable_read_committed
08_nonrepeatable_repeatable_read 09_deadlock; do bash run_concurrent.sh $d; done
bash run_stress_test.sh | tee results/10_stress_test_output.txt
```

Session outputs: `results/<demo>_sessionA.txt`, `results/<demo>_sessionB.txt`, `results/<demo>_final.txt`.

Part C: Fintech operations required by the case study

Demo 11: Reverse a completed transfer (`sql/11_reverse_transfer.sql`)

A committed transaction can never be "un-committed" (**durability**), so a disputed or mistaken transfer is reversed with a new **compensating transaction**. The stored procedure `reverse_transfer()` locks the transfer row and both accounts (lowest account_id first, the same deadlock-prevention rule as `transfer_funds()`), checks that the recipient still has the funds, moves the money back, posts two new ledger rows, marks the transfer `REVERSED` and writes the reason to the audit log.

```
$ grep -v "^--" results/11_reverse_transfer_output.txt | tail -30
        102 | 57500.00
(2 rows)

CALL reverse_transfer(325, 'Customer sent to wrong account');
CALL
SELECT account_id, balance FROM accounts WHERE account_id IN (101,102) ORDER BY 1;
 account_id | balance
        101 | 50000.00
        102 | 50000.00
(2 rows)

SELECT transfer_id, from_account, to_account, amount, status FROM transfers WHERE transfer_id = 325;
 transfer_id | from_account | to_account | amount  |  status
         325 |          101 |        102 | 7500.00 | REVERSED
(1 row)

SELECT account_id, direction, amount FROM transactions WHERE transfer_id = 325 ORDER BY txn_id;
 account_id | direction | amount
        101 | DR        | 7500.00
        102 | CR        | 7500.00
        102 | DR        | 7500.00
        101 | CR        | 7500.00
(4 rows)

CALL reverse_transfer(325, 'second attempt');
psql:sql/11_reverse_transfer.sql:39: ERROR:  Transfer 325 cannot be reversed (status REVERSED)
CONTEXT:  PL/pgSQL function reverse_transfer(bigint,text) line 7 at RAISE
CALL reverse_transfer(99999999, 'does not exist');
psql:sql/11_reverse_transfer.sql:40: ERROR:  Transfer 99999999 does not exist
CONTEXT:  PL/pgSQL function reverse_transfer(bigint,text) line 5 at RAISE
```

The full history is kept (original transfer + reversal) rather than deleting rows, which is what auditors and regulators expect. Note that a transfer that *fails* part-way never needs reversing: atomicity (Demo 2) guarantees

it leaves no trace.

Demo 12: Audit account balances (`sql/12_audit_balances.sql`)

Five double-entry consistency checks. Checks 1–2 should return no rows, check 3 a zero difference, and check 4 zero counts.

Check	What it verifies	My result
1	Every SUCCESS transfer has exactly one DR and one CR of equal amount	**0 rows** (no violations)
2	Every REVERSED transfer nets to zero	**0 rows**
3	Transfers move money but never create or destroy it	credits ₦62,000.00 = debits ₦62,000.00, **difference 0.00**
4	No negative balances, no orphaned accounts	**0 / 0**
5	Audit trail of every balance change on account 101 (who, when, old → new)	shown below

```
$ grep -v "^--" results/12_audit_balances_output.txt | tail -40
 transfer_id | amount | debits | credits
(0 rows)

SELECT t.transfer_id,
       sum(CASE WHEN x.direction='CR' THEN x.amount ELSE -x.amount END) AS net_effect
FROM transfers t JOIN transactions x ON x.transfer_id = t.transfer_id
WHERE t.status = 'REVERSED'
GROUP BY t.transfer_id
HAVING sum(CASE WHEN x.direction='CR' THEN x.amount ELSE -x.amount END) <> 0;
 transfer_id | net_effect
(0 rows)

SELECT sum(amount) FILTER (WHERE direction='CR') AS total_credits,
       sum(amount) FILTER (WHERE direction='DR') AS total_debits,
       sum(amount) FILTER (WHERE direction='CR') - sum(amount) FILTER (WHERE direction='DR') AS difference
FROM transactions WHERE txn_type = 'TRANSFER';
 total_credits | total_debits | difference
      62000.00 |     62000.00 |       0.00
(1 row)

SELECT count(*) FILTER (WHERE balance < 0) AS negative_balances,
       count(*) FILTER (WHERE customer_id NOT IN (SELECT customer_id FROM customers)) AS orphan_accounts
FROM accounts;
 negative_balances | orphan_accounts
                 0 |               0
(1 row)

SELECT audit_id, action, old_data->>'balance' AS old_balance, new_data->>'balance' AS new_balance,
       changed_by, to_char(changed_at,'YYYY-MM-DD HH24:MI:SS') AS changed_at
FROM audit_log WHERE table_name='accounts' AND record_id = 101
ORDER BY audit_id DESC LIMIT 6;
 audit_id | action | old_balance | new_balance | changed_by |     changed_at
      927 | UPDATE | 42500.00    | 50000.00    | postgres   | 2026-11-11 21:37:39
      924 | UPDATE | 50000.00    | 42500.00    | postgres   | 2026-11-11 21:37:39
      922 | UPDATE | 51200.00    | 50000.00    | postgres   | 2026-11-11 21:37:39
      921 | UPDATE | 51100.00    | 51200.00    | postgres   | 2026-11-09 21:27:08
      919 | UPDATE | 51000.00    | 51100.00    | postgres   | 2026-11-09 21:27:08
      916 | UPDATE | 51100.00    | 51000.00    | postgres   | 2026-11-09 21:27:08
(6 rows)
```

The ₦62,000 total matches the transfers made in this portfolio: 320 load-test transfers × ₦100 (₦32,000) + Demo 1 (₦10,000) + Demo 4 (₦5,000) + the ₦7,500 transfer and its reversal (₦15,000). The CHECK constraints, the trigger-based audit log and the ACID transactions together keep the ledger provably consistent even after hundreds of concurrent transfers.

Portfolio 4: NoSQL and Advanced Data Models

Case study: the same PayNaija Wallet data, modelled in three NoSQL families · **Environment:** Windows 11, Docker Desktop (Redis 7, MongoDB 7, Neo4j 5 containers) · **Course link:** Session 1 (NoSQL types, NoSQL vs NewSQL), Session 5 (graph databases and knowledge graphs)

PayNaija keeps **money** in PostgreSQL, because balances need ACID (Portfolios 1–3). Three parts of the workload fit the relational model poorly, and each is moved to the NoSQL model that suits it (**polyglot persistence**):

Workload	Problem in the relational model	NoSQL model chosen
Sessions, PIN brute-force limits, balance cache, live leaderboards	Very high read/write rate; short-lived data that should expire automatically	**Key-value**: Redis
Customer profile + KYC + devices + preferences	Semi-structured; fields differ between customers and change often; needs 4–5 joins to assemble one profile	**Document**: MongoDB
Fraud and anti-money-laundering investigation	Questions are about *relationships* (rings, shared devices, money paths), which need recursive self-joins in SQL	**Graph**: Neo4j

All three databases ran as Docker containers on my laptop. The MongoDB and Neo4j data was **exported from my own Portfolio 1–3 PostgreSQL database** using `export_from_postgres.sh`, so every model describes the same customers, accounts and transfers.

A. Key-value model: Redis

Script: `redis/redis_demo.sh` · Output: `results/redis_demo_output.txt`

Key pattern	Redis type	Purpose	Expiry
`session:<token>`	Hash	Logged-in session (customer, device, IP, KYC tier)	900 s (auto logout)
`pinfail:<customer_id>`	String counter	Brute-force protection: max 3 wrong PINs per 5 min	300 s
`balance:acct:<id>`	String	Cache-aside copy of balance for the app home screen	30 s, deleted on every transfer
`merchants:sales:today`	Sorted set	Real-time merchant leaderboard	daily

```
$ bash redis/redis_demo.sh | tee results/redis_demo_output.txt
redis> HSET session:9f2c1a customer_id 1450 device Android-TECNO-Spark ip 102.89.33.10 kyc_tier 2
4
redis> EXPIRE session:9f2c1a 900
1
redis> HGETALL session:9f2c1a
customer_id
1450
device
Android-TECNO-Spark
ip
102.89.33.10
kyc_tier
2
redis> TTL session:9f2c1a
899
redis> INCR pinfail:1450
attempt 1 -> allowed (count=1)
redis> INCR pinfail:1450
attempt 2 -> allowed (count=2)
redis> INCR pinfail:1450
attempt 3 -> allowed (count=3)
redis> INCR pinfail:1450
attempt 4 -> BLOCKED (count=4)
redis> TTL pinfail:1450
298
redis> SET balance:acct:1450 128450.75 EX 30
OK
redis> GET balance:acct:1450
128450.75
redis> DEL balance:acct:1450
1
redis> GET balance:acct:1450

redis> ZINCRBY merchants:sales:today 15000 Merchant 7
15000
redis> ZINCRBY merchants:sales:today 42000 Merchant 12
42000
redis> ZINCRBY merchants:sales:today 8000 Merchant 3
8000
redis> ZINCRBY merchants:sales:today 30000 Merchant 7
45000
redis> ZREVRANGE merchants:sales:today 0 2 WITHSCORES
Merchant 7
45000
Merchant 12
42000
Merchant 3
8000
```

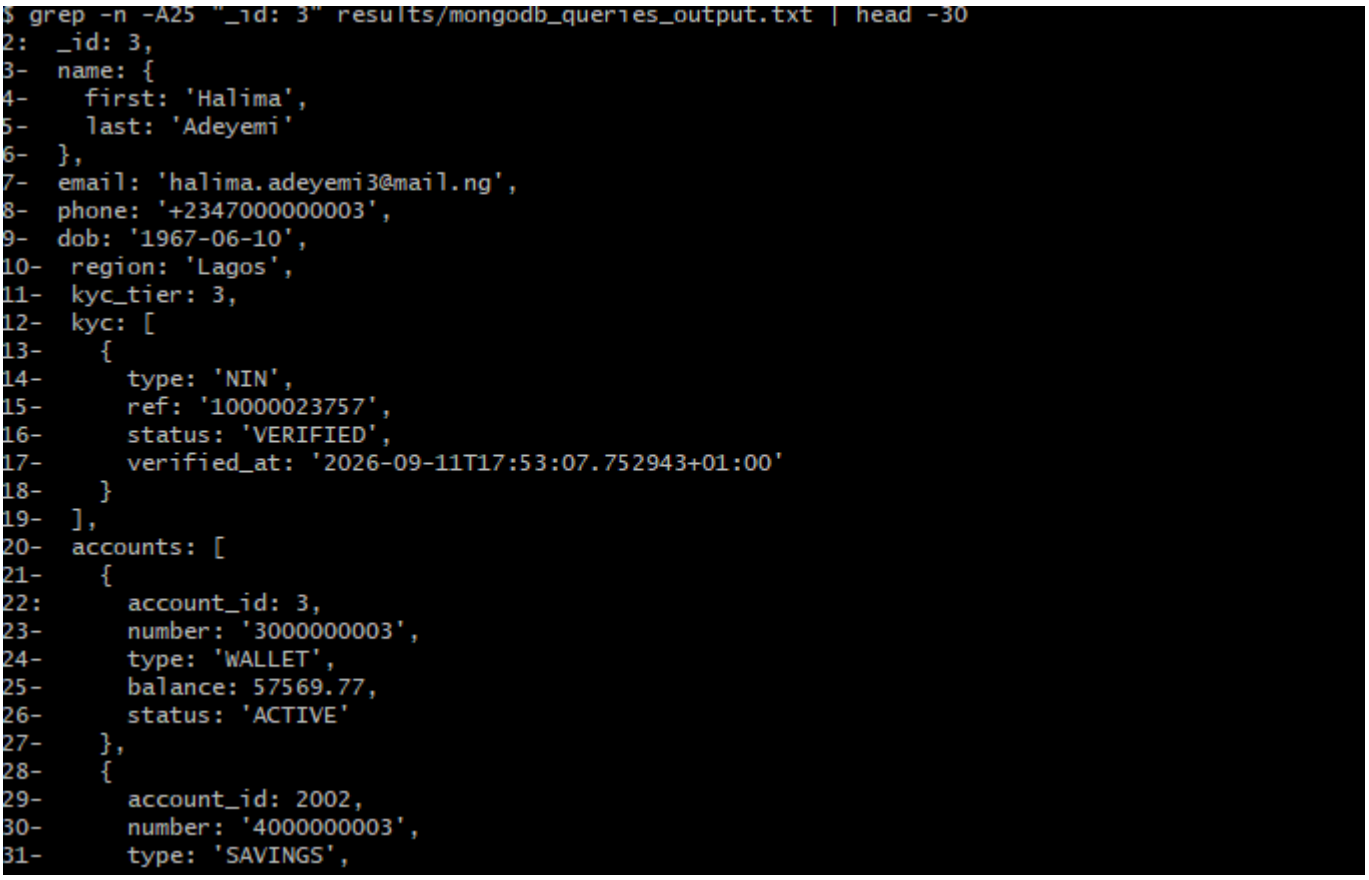
Results: the session was created with a 900-second TTL and was already counting down (`TTL` = 899 s), so Redis will delete it automatically with no clean-up job; PIN attempts 1–3 were allowed and **attempt 4 was BLOCKED** (the counter itself expires after 300 s, `TTL` = 298 s); the cached balance returned `(nil)` once it was invalidated after a transfer; and the sorted set ranked **Merchant 7 (₦45,000)** first.

Why it suits the workload. Every operation is a single O(1) or O(log n) lookup by key, held in memory with sub-millisecond latency. TTL gives automatic expiry, which would need a scheduled cleanup job in SQL. `INCR` is **atomic**, so two simultaneous wrong-PIN attempts cannot both slip through, which matters from a security point of view. Redis gives no joins, no ad-hoc queries and limited durability, so it is used only for data that can be rebuilt. **Balances are never stored only in Redis.**

B. Document model: MongoDB

Files: `mongodb/customers.json` (2,000 documents), `mongodb/transactions.json` (20,000 documents), `mongodb/01_setup.js`, `mongodb/02_queries.js` · Outputs: `results/mongodb_setup_output.txt`, `results/mongodb_queries_output.txt`

Customer document: one read returns the whole profile, which needs 4 tables and 3 joins in PostgreSQL:



```
$ grep -n -A25 "_id: 3" results/mongodb_queries_output.txt | head -30
2:  _id: 3,
3-  name: {
4-    first: 'Halima',
5-    last: 'Adeyemi'
6-  },
7-  email: 'halima.adeyemi3@mail.ng',
8-  phone: '+2347000000003',
9-  dob: '1967-06-10',
10-  region: 'Lagos',
11-  kyc_tier: 3,
12-  kyc: [
13-    {
14-      type: 'NIN',
15-      ref: '10000023757',
16-      status: 'VERIFIED',
17-      verified_at: '2026-09-11T17:53:07.752943+01:00'
18-    }
19-  ],
20-  accounts: [
21-    {
22:      account_id: 3,
23-      number: '3000000003',
24-      type: 'WALLET',
25-      balance: 57569.77,
26-      status: 'ACTIVE'
27-    },
28-    {
29-      account_id: 2002,
30-      number: '4000000003',
31-      type: 'SAVINGS',
```

```
{ _id: 3, name: { first: 'Halima', last: 'Adeyemi' }, email: 'halima.adeyemi3@mail.ng', phone:
'+2347000000003',
  dob: '1967-06-10', region: 'Lagos', kyc_tier: 3,
  kyc: [ { type: 'NIN', ref: '10000023757', status: 'VERIFIED', verified_at: '2026-09-11T17:53:07+01:00' } ],
  accounts: [ { account_id: 3,    number: '3000000003', type: 'WALLET',  balance: 57569.77, status: 'ACTIVE'
},
              { account_id: 2002, number: '4000000003', type: 'SAVINGS', ... } ],
  devices: [ ... ], preferences: { language: 'English' } }
```

Documents in one collection do not need identical fields. For example, every fourth customer has `preferences: { ussd_enabled: true, language: 'Hausa' }`, and `02_queries.js` adds `bvn_linked` and `next_of_kin` to customer 3 only, with no `ALTER TABLE` and no downtime.

Embed vs reference decisions

Data	Decision	Reason
KYC docs, devices, preferences, account summary	**Embed**	Always read with the customer; small and bounded
Transactions	**Reference** (`account_id`) in a separate collection	Unbounded growth: embedding would breach MongoDB's 16 MB document limit and make every profile read slow

Flexibility is balanced by a `$jsonSchema` validator that still enforces email, the Nigerian phone format and the KYC tier range.

Aggregation pipeline: wallet customers and total wallet balance per region (`$unwind → $match → $group → $sort`):

Region	Customers	Total wallet balance (₦)
Lagos	935	228,850,149.39
Oyo	173	44,312,119.74
Kaduna	175	43,755,041.55
Kano	178	43,141,348.68
Rivers	178	41,698,401.07
Enugu	173	40,433,477.38
Abuja FCT	96	25,546,885.20
Anambra	92	22,755,982.26 *(returned as 22755982.259999998)*

Query plan: COLLSCAN vs IXSCAN (`explain("executionStats")` for one account's transactions):

	Stage	Documents examined	Returned	Time
Without index	**COLLSCAN**	**20,000**	7	4 ms
With `{account_id: 1, created_at: -1}`	**IXSCAN → FETCH**	**7**	7	0 ms

```
$ grep -E "totalDocsExamined|nReturned|executionTimeMillis|stage" results/mongodb_queries_output.txt
  nReturned: 7,
  executionTimeMillis: 4,
  totalDocsExamined: 20000,
    stage: 'COLLSCAN',
    nReturned: 7,
    executionTimeMillisEstimate: 0,
  nReturned: 7,
  executionTimeMillis: 0,
  totalDocsExamined: 7,
    stage: 'FETCH',
    nReturned: 7,
    executionTimeMillisEstimate: 0,
      stage: 'IXSCAN',
      nReturned: 7,
      executionTimeMillisEstimate: 0,
```

Without an index MongoDB had to read every document. With the compound index it read exactly the 7 it needed, which is the same principle as the B+ tree indexes in Portfolio 2.

Finding: floating-point money. Anambra's total came back as **22755982.259999998**. The balances were imported from JSON as binary floating-point numbers (`double`), which cannot represent most decimal amounts exactly, so summing them introduced a rounding error. The same data in PostgreSQL is stored as `NUMERIC(15,2)` and sums exactly. This is direct evidence for the Portfolio 1 design decision (never use FLOAT for money). In production, MongoDB money fields should use the **`Decimal128`** type.

C. Graph model: Neo4j (fraud detection)

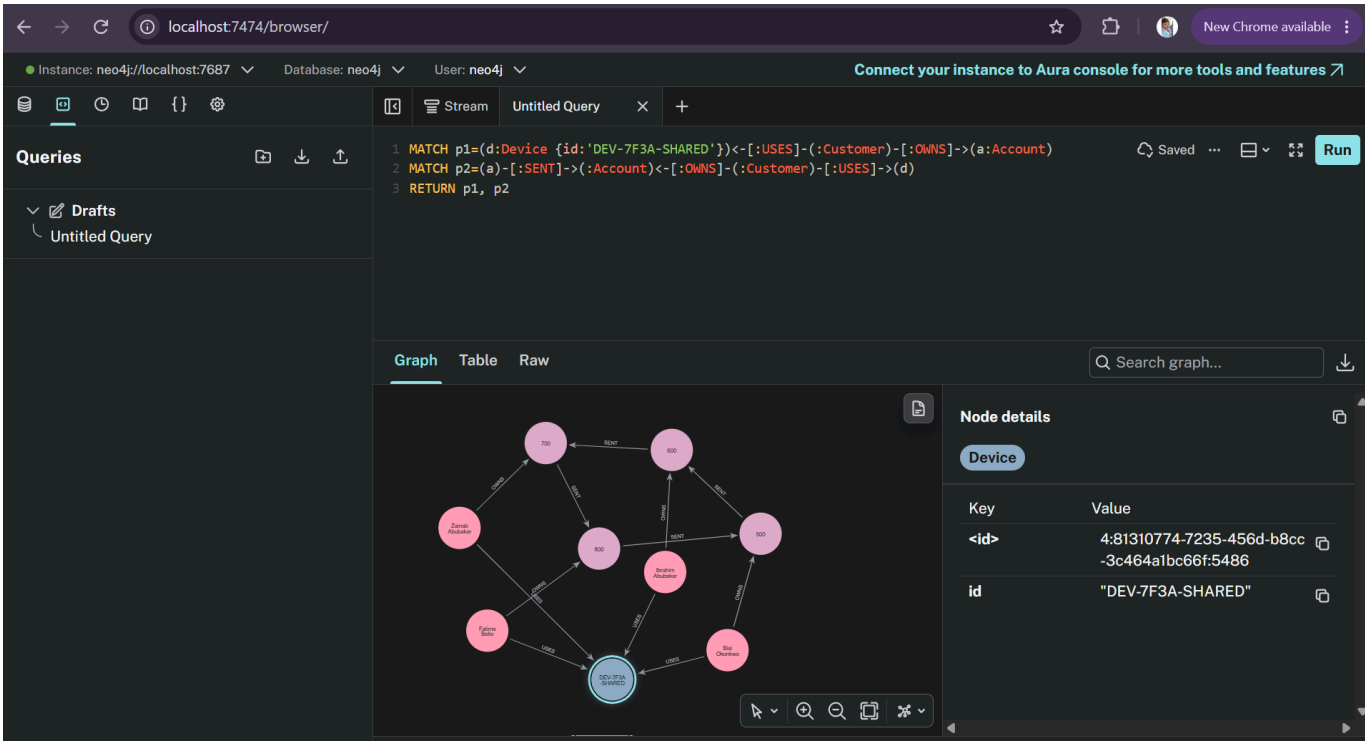
Files: `neo4j/00_generate_transfers.sql`, CSV exports (`customers.csv`, `accounts.csv`, `devices.csv`, `transfers.csv`), `neo4j/01_load_graph.cypher`, `neo4j/02_fraud_queries.cypher` · Output: `results/neo4j_fraud_output.txt`

Graph model (property graph)

```
(:Customer {id, name, region}) -[:OWNS]->  (:Account {id, type})
(:Customer)                    -[:USES]->  (:Device {id})
(:Account)                     -[:SENT {amount, at}]-> (:Account)
```

Transfer activity was generated in my PostgreSQL through `transfer_funds()`, including a planted laundering pattern. The accounts of four customers who all log in from **one device** pass money in a circle.

The fraud ring found by Neo4j:



Cypher queries and my results (`02_fraud_queries.cypher`):

```
// Q1 circular money flow: cycles of 3-5 transfers back to the start
MATCH path = (a:Account)-[:SENT*3..5]->(a)
RETURN [n IN nodes(path) | n.id] AS ring, [r IN relationships(path) | r.amount] AS amounts;
```

```
ring                    amounts
[500, 600, 700, 800, 500] [250000.0, 245000.0, 240000.0, 235000.0]
[600, 700, 800, 500, 600] [245000.0, 240000.0, 235000.0, 250000.0]
... (the same ring found from each of its 4 starting accounts)

Q2 shared device:  "DEV-7F3A-SHARED", 4, ["Ibrahim Abubakar", "Zainab Abubakar", "Bisi Okonkwo", "Fatima
Bello"]
Q3 device + money: Bisi Okonkwo -> Ibrahim Abubakar -> Zainab Abubakar -> Fatima Bello -> Bisi Okonkwo
Q4 money path (customer 500 to customer 799): [500, 600, 700, 800]
```

```
$ cat results/neo4j_fraud_output.txt
ring, amounts
[500, 600, 700, 800, 500], [250000.0, 245000.0, 240000.0, 235000.0]
[600, 700, 800, 500, 600], [245000.0, 240000.0, 235000.0, 250000.0]
[700, 800, 500, 600, 700], [240000.0, 235000.0, 250000.0, 245000.0]
[800, 500, 600, 700, 800], [235000.0, 250000.0, 245000.0, 240000.0]
device, customers_sharing, customers
"DEV-7F3A-SHARED", 4, ["Ibrahim Abubakar", "Zainab Abubakar", "Bisi Okonkwo", "Fatima Bello"]
device, sender, receiver
"DEV-7F3A-SHARED", "Bisi Okonkwo", "Ibrahim Abubakar"
"DEV-7F3A-SHARED", "Ibrahim Abubakar", "Zainab Abubakar"
"DEV-7F3A-SHARED", "Zainab Abubakar", "Fatima Bello"
"DEV-7F3A-SHARED", "Fatima Bello", "Bisi Okonkwo"
money_path
[500, 600, 700, 800]
```

Interpretation. Two independent signals point at the same four customers: (1) money travels in a closed loop, and (2) all four "different" customers use the same device. The amounts also fall by **₦5,000 at every hop** (₦250,000 → ₦245,000 → ₦240,000 → ₦235,000), the classic *layering* pattern in which each mule keeps a cut before passing the money on. PayNaija's compliance team would freeze the four accounts and file a suspicious transaction report.

Why a graph suits this. In SQL, finding a 3–5-hop cycle needs a recursive CTE that self-joins `transfers` repeatedly; the cost grows with every hop and with the table size. Neo4j stores relationships as direct pointers

(*index-free adjacency*), so a traversal costs time in proportion to the relationships actually followed, not the total data size. The pattern `(a)-[:SENT*3..5]->(a)` also states the business question almost word for word. The trade-offs noted in Session 5 apply: a separate query language (no single standard: Cypher vs SPARQL vs Gremlin), harder horizontal scaling, and data integration (here an export from PostgreSQL).

D. Comparison and conclusion

Criterion	PostgreSQL (relational)	Redis (key-value)	MongoDB (document)	Neo4j (graph)
Schema	Fixed, enforced	None	Flexible + optional validator	Flexible labels/properties
Query style	SQL, joins	GET/SET by key	Filters, aggregation pipeline	Pattern matching (Cypher)
Consistency model	ACID	Atomic single-key operations	ACID per document (multi-document transactions available)	ACID (single instance)
Scaling	Vertical + read replicas	Horizontal (cluster)	Horizontal (sharding)	Mostly vertical
Money precision	Exact (`NUMERIC`)	Strings only	`double` by default (rounding seen) / `Decimal128`	Float properties
Best PayNaija use	Balances, ledger, transfers	Sessions, rate limits, cache	Customer/KYC profiles	Fraud rings, investigations

NoSQL vs NewSQL. If PayNaija outgrew a single PostgreSQL primary for the *ledger itself*, the answer would not be NoSQL, because giving up ACID on money is unacceptable (the floating-point rounding above is a small example of the risk). It would be a **NewSQL** database such as CockroachDB or Google Spanner, which keeps SQL and ACID while scaling horizontally (Session 1).

How to run (Windows, Docker Desktop, UCRT64 terminal)

```
# Data from PostgreSQL
psql -d paynaija -f neo4j/00_generate_transfers.sql
bash export_from_postgres.sh
export MSYS_NO_PATHCONV=1

# Redis
docker run -d --name paynaija-redis -p 6379:6379 redis:7
bash redis/redis_demo.sh | tee results/redis_demo_output.txt

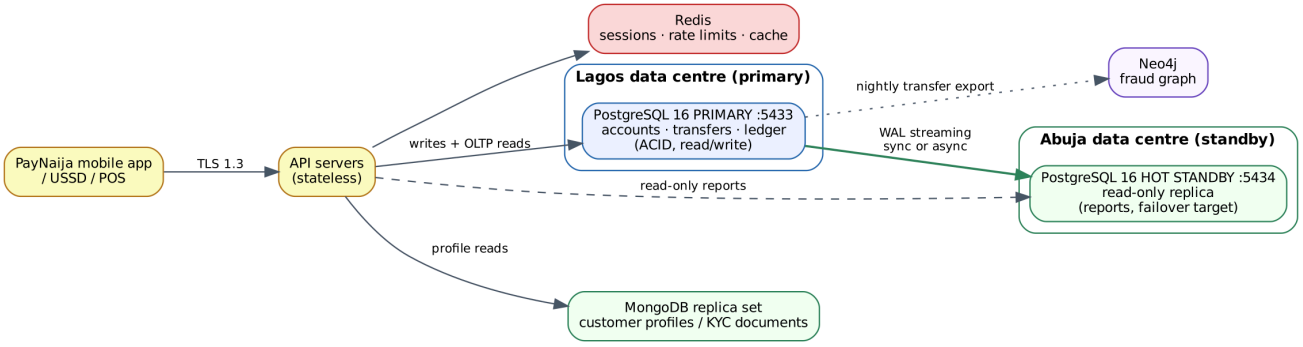
# MongoDB
docker run -d --name paynaija-mongo -p 27017:27017 mongo:7
docker cp mongodb/. paynaija-mongo:/data/import/
docker exec paynaija-mongo mongoimport --db paynaija --collection customers    --jsonArray --file
/data/import/customers.json
docker exec paynaija-mongo mongoimport --db paynaija --collection transactions --jsonArray --file
/data/import/transactions.json
docker exec paynaija-mongo mongosh paynaija --quiet /data/import/01_setup.js
docker exec paynaija-mongo mongosh paynaija --quiet /data/import/02_queries.js

# Neo4j (browser: http://localhost:7474)
docker run -d --name paynaija-neo4j -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/<password> \
       -v "$(pwd -W)/neo4j:/var/lib/neo4j/import" neo4j:5
docker exec paynaija-neo4j cypher-shell -u neo4j -p <password> -f /var/lib/neo4j/import/01_load_graph.cypher
docker exec paynaija-neo4j cypher-shell -u neo4j -p <password> -f
/var/lib/neo4j/import/02_fraud_queries.cypher
```

Portfolio 5: Distributed and Cloud Database Exercise

Case study: PayNaija Wallet across two data centres, Lagos (primary) and Abuja (standby) · **DBMS:** PostgreSQL 16 streaming replication in Docker Desktop (Windows 11) · **Course link:** Study Session 2 (DDBS architecture, fragmentation, replication, consistency models, CAP, ACID vs BASE) and Session 3 (cloud databases)

1. Architecture



Lagos runs the read/write primary; Abuja runs a hot standby that receives the write-ahead log (WAL) continuously. Reporting queries can be sent to the standby, which takes load off the primary.

This is a **homogeneous** distributed database (same DBMS and OS at both sites) in a **primary–standby (client-server)** arrangement. It gives **replication transparency**: applications connect to "the database" and do not manage copies themselves.

2. Setting up replication (Docker Desktop on Windows 11)

The two sites run as two PostgreSQL 16 containers defined in `docker/docker-compose.yml`:

Container	Role	Host port
`paynaija_lagos_primary`	Read/write primary (`wal_level=replica`)	5433
`paynaija_abuja_replica`	Hot standby created with `pg_basebackup -R -X stream -C -S abuja_slot`	5434

`docker/init-primary.sh` creates a `replicator` role with the REPLICATION privilege and allows it in `pg_hba.conf`. The replica clones the primary, writes `standby.signal` + `primary_conninfo` (application_name `abuja_standby`), and uses a **replication slot**, so the primary keeps the WAL the replica still needs. My PayNaija database (Portfolios 1–4) was then loaded into Lagos with `pg_dump | psql`. Because the replica was already streaming, the whole load was replicated to Abuja automatically.

```
cd docker && docker compose up -d
pg_dump -d paynaija --no-owner --no-privileges | PGPASSWORD=paynaija psql -h localhost -p 5433 -U postgres -d
paynaija
```

Evidence (`scripts/02_replication_demo_docker.sh` → `results/02_replication_demo_output.txt`):

```
### [PRIMARY - Lagos :5433] connected standbys
application_name | abuja_standby
state            | streaming
sync_state       | async
sent_lsn         | 0/4A3E900
replay_lsn       | 0/4A3E900          <- standby has replayed everything sent
replay_lag       | 00:00:00.002417

### [REPLICA - Abuja :5434] am I a standby?
 is_standby | replayed_up_to
 t          | 0/4A3E900

### [PRIMARY] register a new customer       -> customer_id 2007 ('Ada', 'Replica')
### [REPLICA] the same row is already readable on the standby
 customer_id | first_name | last_name |        email
        2007 | Ada        | Replica   | ada.replica@mail.ng

### [REPLICA] writes are rejected (read-only hot standby)
ERROR:  cannot execute UPDATE in a read-only transaction

primary  : 2001 customers, 201438 transactions
replica  : 2001 customers, 201438 transactions
replication_delay  00:00:02.025339
```

```
$ bash scripts/02_replication_demo_docker.sh 2>&1 | tee results/02_replication_demo_output.txt
### [PRIMARY - Lagos :5433] connected standbys
SELECT application_name, state, sync_state, sent_lsn, replay_lsn, replay_lag FROM pg_stat_replication;
-[ RECORD 1 ]----+----------------
application_name | abuja_standby
state            | streaming
sync_state       | async
sent_lsn         | 0/4A3E900
replay_lsn       | 0/4A3E900
replay_lag       | 00:00:00.002417

### [REPLICA - Abuja :5434] am I a standby?
SELECT pg_is_in_recovery() AS is_standby, pg_last_wal_replay_lsn() AS replayed_up_to;
 is_standby | replayed_up_to
------------+----------------
 t          | 0/4A3E900
(1 row)

### [PRIMARY] register a new customer
INSERT INTO customers(first_name,last_name,email,phone,date_of_birth,region_id,kyc_tier)
        VALUES ('Ada','Replica','ada.replica@mail.ng','+2348090000001','1995-03-15',2,2) RETURNING customer_id, created_at;
 customer_id |          created_at
-------------+-------------------------------
        2007 | 2026-11-11 23:43:47.708383+00
(1 row)

INSERT 0 1
### [REPLICA] the same row is already readable on the standby
SELECT customer_id, first_name, last_name, email, created_at FROM customers WHERE email='ada.replica@mail.ng';
 customer_id | first_name | last_name |        email        |          created_at
-------------+------------+-----------+---------------------+-------------------------------
        2007 | Ada        | Replica   | ada.replica@mail.ng | 2026-11-11 23:43:47.708383+00
(1 row)

### [REPLICA] writes are rejected (read-only hot standby)
UPDATE accounts SET balance = 0 WHERE account_id = 1;
ERROR:  cannot execute UPDATE in a read-only transaction
### Row counts match on both sites
primary  : 2001 customers, 201438 transactions
replica  : 2001 customers, 201438 transactions
### [REPLICA] replication delay
SELECT now() - pg_last_xact_replay_timestamp() AS replication_delay;
 replication_delay
-------------------
 00:00:02.025339
(1 row)
```

The `replay_lag` of **2.4 ms** is the real replication delay. The final `replication_delay` (2 s) only measures the time since the *last* transaction was replayed: the script had been idle for about 2 seconds, so it is not lag.

3. Consistency vs availability: the CAP theorem in practice

`scripts/03_cap_sync_vs_async_docker.sh`. A **network partition** is simulated by stopping the Abuja container (`docker stop paynaija_abuja_replica`), and a customer then deposits ₦5,000 in Lagos.

(a) Synchronous replication (`synchronous_standby_names = 'abuja_standby'`): a commit returns only after Abuja confirms it has the data.

```
 application_name | sync_state
 abuja_standby    | sync
-- Abuja unreachable; customer deposits N5,000 --
after 5 seconds the COMMIT has still not returned - the session is waiting for the standby:
 pid | state  | wait_event_type | wait_event | waiting_for
 752 | active | IPC             | SyncRep    | 00:00:05.140048
operator cancels the wait:
WARNING:  canceling wait for synchronous replication due to user request
DETAIL:  The transaction has already committed locally, but might not have been replicated to the standby.
```

→ **CP behaviour:** during the partition the system stays **consistent** (no write is acknowledged unless it exists in both data centres) but loses **availability** (the customer's deposit hangs). The DETAIL line is an important subtlety: the cancelled transaction *was* committed in Lagos. Only the acknowledgement to the client was withheld, which is why production systems pair synchronous replication with a quorum of more than one standby.

(b) Asynchronous replication (`synchronous_standby_names = ''`), same partition:

```
UPDATE 1
Time: 5.466 ms
```

→ **AP-leaning behaviour:** the write succeeds immediately (**available**), but Abuja does not have it yet. If Lagos were destroyed at this moment, this deposit could be lost. Consistency is only **eventual**.

(c) Partition heals: `docker start` brings the replica back; it reconnects through its replication slot and catches up automatically.

```
 primary_balance 83818.09  |  replica_balance 83818.09  |  abuja_standby  streaming  async
```

```
$ bash scripts/03_cap_sync_vs_async_docker.sh 2>&1 | tee results/03_cap_sync_vs_async_output.txt
### STEP 1 - switch to SYNCHRONOUS replication (a commit waits until Abuja confirms)
 application_name | sync_state
------------------+------------
 abuja_standby    | sync
(1 row)

### STEP 2 - partition: the Abuja replica becomes unreachable
paynaija_abuja_replica
### STEP 3 - a customer tries to deposit N5,000 on the Lagos primary
after 5 seconds the COMMIT has still not returned - the session is waiting for the standby:
 pid | state  | wait_event_type | wait_event |   waiting_for   |                          query
-----+--------+-----------------+------------+-----------------+-----------------------------------------------------------
 752 | active | IPC             | SyncRep    | 00:00:05.140048 | UPDATE accounts SET balance = balance + 5000 WHERE account_i
(1 row)

operator cancels the wait:
WARNING:  canceling wait for synchronous replication due to user request
DETAIL:  The transaction has already committed locally, but might not have been replicated to the standby.
UPDATE 1
=> CONSISTENCY chosen over AVAILABILITY (CP): the system would rather block than acknowledge
   a write that exists in only one data centre.
### STEP 4 - switch to ASYNCHRONOUS replication and repeat the write during the partition
Timing is on.
UPDATE 1
Time: 5.466 ms
=> AVAILABILITY chosen (AP-style): the write succeeds immediately, but if Lagos failed now
   before Abuja reconnects, this deposit could be lost (eventual consistency).
### STEP 5 - partition heals: replica restarts and catches up automatically
paynaija_abuja_replica
 primary_balance
-----------------
        83818.09
(1 row)

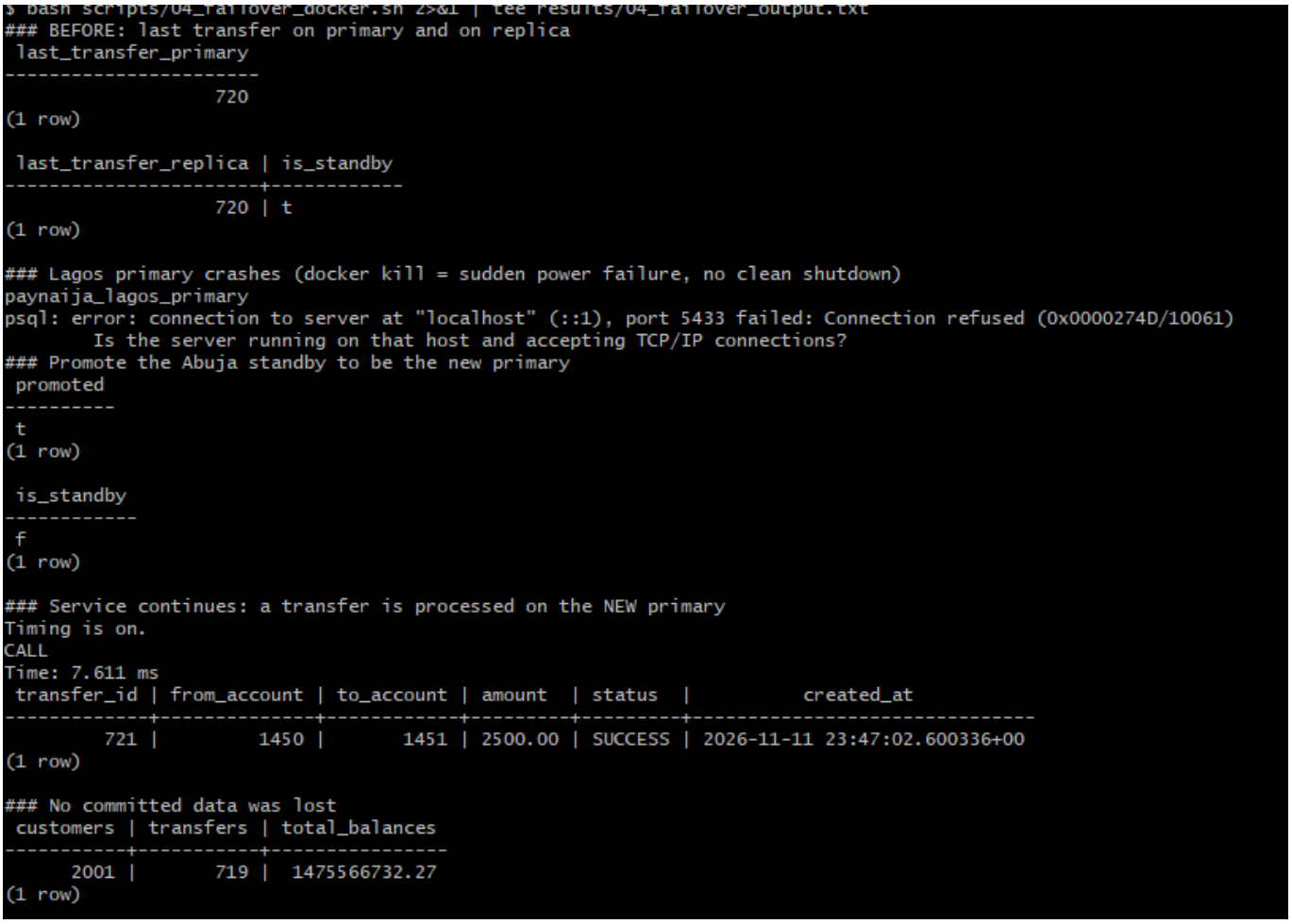
 replica_balance
-----------------
        83818.09
(1 row)

 application_name |   state   | sync_state
------------------+-----------+------------
 abuja_standby    | streaming | async
(1 row)
```

4. High availability: failover

`scripts/04_failover_docker.sh`. The Lagos primary is killed with `docker kill` (SIGKILL, equivalent to a sudden power failure with no clean shutdown) and Abuja is promoted.

```
 last_transfer_primary 720   |   last_transfer_replica 720, is_standby t
 ### Lagos primary crashes
 psql: error: connection to server at "localhost" (::1), port 5433 failed: Connection refused
 ### Promote the Abuja standby
  promoted | t           is_standby | f
 ### Service continues on the NEW primary
 CALL transfer_funds(1450, 1451, 2500);   -> CALL   Time: 7.611 ms
  transfer_id | from_account | to_account | amount  | status
          721 |         1450 |       1451 | 2500.00 | SUCCESS
 ### No committed data was lost
  customers 2001 | transfers 719 | total_balances 1475566732.27
```



Every transfer committed before the crash (up to id 720) exists on the new primary, so the **durability** of committed transactions survived the loss of a whole site. (There are 719 rows but the highest id is 720 because transfer id 1 was consumed by a constraint test in Portfolio 1 that was rejected. Sequences are never rolled back.) In my run the next id was 721 with no gap; PostgreSQL pre-logs sequence values in batches, so after some failovers a gap of a few ids can appear, which is also not data loss.

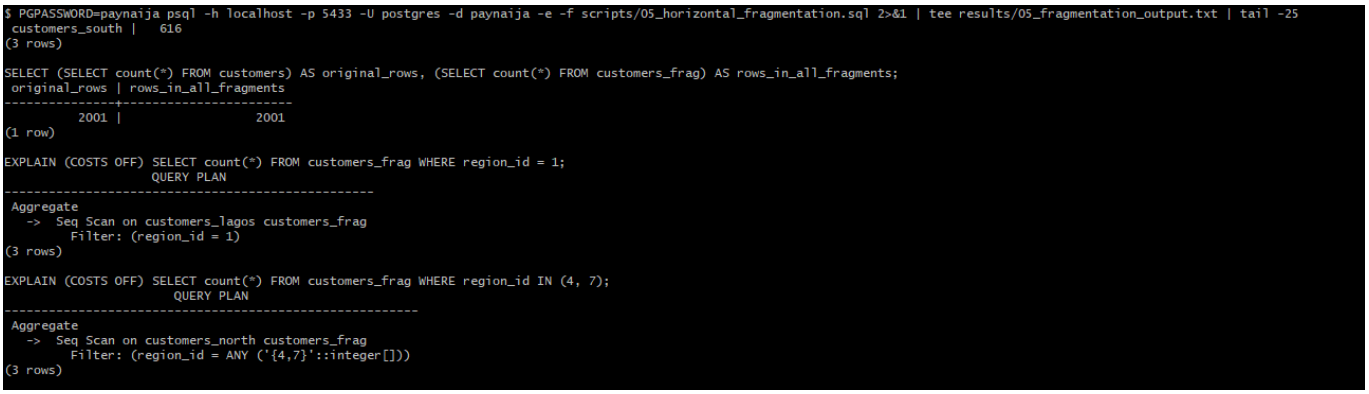
What a real deployment adds: promotion here was manual (`pg_promote()`). In production a failover manager (e.g. Patroni, or the cloud provider's Multi-AZ automation) detects the failure, promotes the standby and moves a virtual IP/DNS name so applications reconnect automatically. It also **fences** the old primary so it cannot come back as a second primary (*split brain*).

5. Horizontal fragmentation (sharding design)

`scripts/05_horizontal_fragmentation.sql` fragments `customers` by region using list partitioning (run on the Lagos primary):

```
     fragment     | count            original_rows | rows_in_all_fragments
  customers_lagos |   935                     2001 |                 2001   <- completeness
  customers_north |   450   (Abuja, Kano, Kaduna)
  customers_south |   616   (Rivers, Enugu, Oyo, Anambra)

 EXPLAIN SELECT count(*) FROM customers_frag WHERE region_id = 1;
  Aggregate ->  Seq Scan on customers_lagos           <- only the Lagos fragment is read
 EXPLAIN SELECT count(*) FROM customers_frag WHERE region_id IN (4, 7);
  Aggregate ->  Seq Scan on customers_north           <- only the North fragment is read
```



The fragments are **complete** (every row is in some fragment), **disjoint** (no row is in two), and **reconstructable** (the union equals the original). The planner's *partition pruning* gives **fragmentation transparency**: the query names `customers_frag` and never the fragment.

Sharding plan for growth. Here the fragments are on one server. At scale each fragment would sit on its own node (e.g. with the Citus extension or a NewSQL database such as CockroachDB):

Option	Shard key	Pros	Cons
By region (list)	`region_id`	Data stays near users; supports NDPA in-country residency; easy to explain	**Lagos holds 47% of customers, creating a hot shard**
By hash	`hash(customer_id)`	Even spread across N shards	Regional reports must query every shard
Chosen: hash on `customer_id`, with accounts and transactions co-located on the same shard		A customer's transfers between their own accounts stay single-shard; even load	Transfers between customers on different shards need a distributed transaction (**two-phase commit**), which adds latency

6. Cloud deployment design

Concern	Design for PayNaija on a managed cloud database (e.g. Amazon RDS for PostgreSQL / Azure Database for PostgreSQL)
Availability	Multi-AZ deployment: a synchronous standby in a second availability zone with automatic failover (the same mechanism tested in Sections 3–4, managed by the provider)
Read scaling	1–2 asynchronous read replicas for statements and reports
Backup / DR	Automated daily snapshots + point-in-time recovery from WAL; cross-region snapshot copy
Security	Encryption at rest (AES-256, KMS-managed keys), TLS in transit, private subnet with no public IP, IAM/RBAC, audit logging (Session 3 & 6)
Compliance	NDPA 2023 requires safeguards for cross-border transfer of Nigerians' personal data, so the primary would be placed in an in-country or approved region, with a DPIA before migration
Cost model	Pay-as-you-go; scale compute up for month-end peaks; serverless options for dev/test

7. Analysis: which consistency model for which data?

PayNaija data	Requirement	Choice
Balances, transfers, ledger	Must never be lost or diverge	**ACID + synchronous replication (CP)**: a brief outage is better than wrong money
Statements, reports on replicas	Can be a second behind	Asynchronous replicas (read-your-writes is not guaranteed on a replica, so a customer's own new transfer is read from the primary)
Notifications, leaderboards, recommendations	Availability first	**BASE / eventual consistency (AP)** in Redis/MongoDB

PostgreSQL's single-primary design is a CP-leaning system. The experiment showed that the CAP trade-off is not fixed by the product: it was **configured** with one setting (`synchronous_standby_names`), and both behaviours were observed on the same database.

How to run (Windows, Docker Desktop, UCRT64 terminal)

```
cd docker && docker compose up -d && cd ..
pg_dump -d paynaija --no-owner --no-privileges > dump.sql
PGPASSWORD=paynaija psql -h localhost -p 5433 -U postgres -d paynaija -q -f dump.sql && rm dump.sql
bash scripts/02_replication_demo_docker.sh
PGPASSWORD=paynaija psql -h localhost -p 5433 -U postgres -d paynaija -e -f
scripts/05_horizontal_fragmentation.sql
bash scripts/03_cap_sync_vs_async_docker.sh
bash scripts/04_failover_docker.sh          # run last: it kills the primary
# reset everything: cd docker && docker compose down -v
```

`scripts/01_setup_replica.sh`, `02_replication_demo.sh`, `03_cap_sync_vs_async.sh` and `04_failover.sh` are the equivalent native-Linux versions of the same exercise.

AI Use Declaration: MIT 8103 CA

Student: Amazobi Amarachi · **Student ID:** 301919110

Item	Declaration
AI tool used	Claude (Anthropic)
What it was used for	Summarising the course text for revision; planning the portfolio structure; drafting the SQL, shell, MongoDB and Cypher scripts; adapting the scripts to my Windows/Docker environment; explaining execution plans and error messages while I worked; drafting the README text, the technical reflection and the documentation from my own results and experiences.
How I reviewed and verified it	I installed PostgreSQL, Docker, Redis, MongoDB and Neo4j on my own laptop and ran every script myself. All results, timings, execution plans and screenshots in this portfolio come from my machine, captured as I ran them and committed progressively to GitHub. I checked each output against what the theory predicted (for example the lost-update balance, the deadlock, the CAP behaviour) and corrected every README so that it reports my own numbers, not example values. I read and edited the drafted text, including the reflection, so that it describes what actually happened when I did the work, and I can explain every script, design decision and result.